

Habib University
CSE 351 – Artificial Intelligence
Fall 2026
Assignment 01
100 Points

Question 1 – Problem Solving via Search **[60 Points]**

You have to develop a generic implementation of search algorithms that can solve a variety of search problems. The task is divided into the following parts.

a) Framing the Problems **[30 points (15+15)]**

An interface of Search Problems is provided to you (`search.py`) in the form of an abstract base class in Python which contains the following functions:

- `getStartState`
- `isGoalState`
- `getSuccessors`
- `getCostOfActions`
- `getHeuristic`

You have to formulate the following two problems as Search Problems by implementing the given interface for both of them.

Problem I – Robot Navigation with Obstacles

You are given a 2-D grid representing an environment in which a robot has to navigate from a specified starting position to a specified goal position.

The grid consists of:

- Free cells that the robot can move through.
- Obstacles that the robot cannot enter.
- A starting cell indicating the robot's initial position.
- A goal cell indicating the destination.

The robot can move between adjacent cells in the grid. At each step, the robot can move up, down, left, or right, provided that the destination cell is within the grid and is not occupied by an obstacle.

Your task is to formulate this problem as a Search Problem by implementing the provided interface.

Your formulation should clearly define:

1. The representation of a state.
2. The initial state.
3. The goal state.
4. The possible actions.

5. The successor function.
6. The cost associated with each action.
7. An appropriate heuristic function for A* Search.

The objective is to find a path from the starting position to the goal while avoiding all obstacles.

For example, a grid may be represented as follows:

<i>S</i>	.	.	#	.	.	.
#	#	.	#	.	#	.
.	.	.	.	.	#	.
.	#	#	#	.	.	.
.	.	.	.	.	#	<i>G</i>

where:

- *S* represents the starting position.
- *G* represents the goal position.
- # represents an obstacle.
- . represents a free cell.

The implementation should work with different grid sizes, starting positions, goal positions, and obstacle configurations.

Problem II – Courier Delivery Route Planning

You work as a route-planning developer for a courier company operating in Karachi. Deliveries are dispatched from a central hub in **Saddar**, and riders/drivers need to reach delivery points across the city as efficiently as possible.

Your program will take the following CSV files as inputs:

- i. **Connections.csv** – the road network mentioning the areas of Karachi that are directly connected to each other, with their respective road distances in km (−1 represents no direct connection).
- ii. **heuristics.csv** – aerial (straight-line) distance between every two areas.
- iii. **TrackType.csv** – the type of road on each connection: **M** – Main Road/Highway, **S** – Standard city road, **N** – Narrow lane (too tight for a delivery van; requires switching to a bike courier) (−1 represents no connection).

Given a starting location and a destination delivery address, you have to find the shortest route between them. The company wants deliveries to travel on main roads as much as possible and avoid narrow lanes wherever it can, since a van rerouted through a narrow lane loses time waiting for a bike courier handoff.

Your formulation should clearly define:

1. The representation of a state.
2. The initial state.
3. The goal state.

4. The possible actions.
5. The successor function.
6. The cost associated with each action, including how road types affect the cost.
7. An appropriate heuristic function for A* Search.

b) Solving the Search Problems

[10 points]

Develop a generic search agent that takes a Search Problem and returns its solution using:

- i. A* Search
- ii. Dijkstra Search (You may use your implementation from CS 102 – DSA.)

Your search agent should be generic enough to operate on both Problem I (Robot Navigation) and Problem II (Courier Delivery Route Planning) through the provided Search Problem interface.

For each search problem and search algorithm, your implementation should report:

- The solution path.
- The total cost of the solution.
- The number of nodes/states visited or expanded.

c) Comparative Analysis

[10 points]

Perform a comparison between A* Search and Dijkstra Search.

For both Robot Navigation and Courier Delivery Route Planning, test the two search techniques using several appropriate start/destination configurations.

Your comparison should include:

- Length/cost of the optimal path.
- Number of nodes/states visited or expanded.
- The resulting route.
- Effect of the heuristic on the performance of A* Search.

You can present your comparison in the form of tables or graphs showing the performance of the two search techniques for various input configurations.

d) Route with Stopovers

[10 points]

The delivery van has several packages to drop off in one run before returning to the hub. Adapt your A* implementation so that the planned route includes **all specified stopovers** before returning to the hub.

Write pseudocode for your modified implementation.

Your solution should:

1. Accept the starting hub and a list of delivery stopovers.
2. Determine an efficient order in which the stopovers should be visited.

3. Use A* Search to find a route between consecutive locations.
4. Return to the hub after completing all deliveries.

The final output should include the complete route, total route cost, and the order in which the stopovers were visited.

Question 2 – Optimization

[30 Points]

a) Function Optimization using Simulated Annealing

[20 points]

A manufacturing company wants to optimize the performance of a system whose performance can be represented mathematically as a function of two variables, x and y .

Implement the Simulated Annealing algorithm to find if there exists a global maximum/minimum of a given function. You will begin with the following parameter values and will fine-tune them during testing:

- Neighborhood size: 0.5
- Starting value of T : 1 (decaying by 0.1 after every K iterations)
- Number of iterations to run for a given temperature (K): 100

The following functions will be used as examples:

Booth Function

$$f(x, y) = (x + 2y - 7)^2 + (2x + y - 5)^2$$

$$-10 \leq x, y \leq 10$$

Himmelblau Function

$$f(x, y) = (x^2 + y - 11)^2 + (x + y^2 - 7)^2$$

$$-5 \leq x, y \leq 5$$

Griewank Function

$$f(x, y) = 1 + \frac{x^2 + y^2}{4000} - \cos(x) \cos\left(\frac{y}{\sqrt{2}}\right)$$

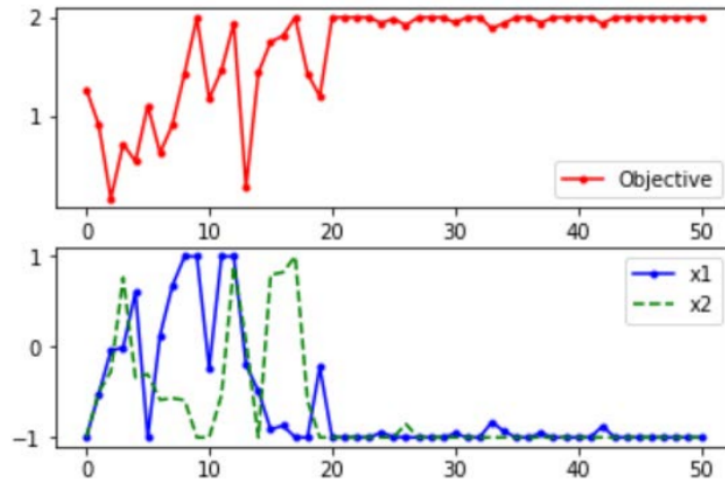
$$-30 < x < 30, \quad -30 < y < 30$$

b) Plotting Graphs

[10 points]

You are required to give visibility of the execution of your Simulated Annealing algorithm by plotting graphs of x , y , and f over iterations as shown below.

You can use the matplotlib library in Python to plot these graphs.



Submission Instructions

Additional Assignment Instructions

1. Good Coding Practices and Rough Work

[10 Points]

A total of 10 points will be allocated for demonstrating good programming and problem-solving practices. Students are expected to:

- Follow good coding practices, including clear and meaningful variable and function names, appropriate code organization, proper indentation, and sufficient comments.
- Show their rough work and problem-solving process used to arrive at their solutions.
- Perform the rough work by hand on paper and submit scanned copies of these pages along with the assignment.

The submitted rough work should demonstrate the reasoning, intermediate steps, and calculations used while developing the solutions.

2. Submission Format

The assignment must be submitted on Canvas as a single ZIP file. The ZIP file must contain separate code files for both questions, along with the scanned copies of the students' handwritten rough work.

3. Late Submission Policy

The late submission policy specified in the course syllabus will apply to this assignment. Late submissions will be accepted for up to two days after the stated deadline and will incur a penalty of 10%.

4. Pair Submission

This assignment must be completed and submitted in pairs. Each pair should submit a single joint submission on Canvas. Both students in the pair are expected to contribute to the submitted work and should be able to explain their implementation and solutions.